

A Founder Letter

Forge — a design-time engine for modern game development

Designer-First · Deterministic · Engine-Agnostic

Opening Note

Forge did not begin as a product idea.

It began as a pattern I could not ignore.

Across small teams, modest projects, and eventually the larger game industry from a distance, I kept seeing the same pattern: game development repeatedly rebuilds its own foundations. Every new project creates new tools, new workflows, new systems, new debugging problems, and new dependencies before the team can get to the experience it actually wants to build.

The problem was not talent.

The problem was not ambition.

The problem was the pipeline.

Forge exists because game development still starts too close to zero.

01 / The First Time I Saw the Problem

The first time I saw the pipeline break was in college.

I entered a game development competition that required one business student, two software engineers, and one artist. I was the business student, studying Supply Chain Management, so my role was to help steer the project: coordinate the team, keep the work moving, and make sure we had something playable by the deadline.

Our game was not a massive production. It was a simple Android game called *Eco Man* — a Frogger-inspired educational game.

But even at that tiny scale, progress felt far harder than it should have.

It took a surprising amount of time just to make something playable. Not finished. Not polished. Just playable.

Character movement took time. Collision took time. Asset integration took time. Every small piece had to be built, revised, debugged, and connected to every other piece. The artist reworked assets. The programmers revised systems. Deadlines slipped. Pivots became constant.

At the time, I thought that was just what early game development looked like.

Later, I realized I had seen a small version of a very large problem.

02 / The Pattern Repeated

Years later, I ran a small technology operation in China.

Different team.

Different country.

Different project.

Same pattern.

Programmers built bespoke systems. Those systems changed. New constraints appeared. Features became harder to support. Artists reworked assets. Progress depended on specific people being available. When key contributors were delayed, the whole project slowed.

The scale had changed, but the pipeline had not.

That was the moment I stopped seeing the problem as a team issue.

It was structural.

03 / The Realization

What struck me was not that games are difficult to build.

Games should be difficult. They are complex creative systems.

What struck me was how often game development begins by rebuilding the same categories of work:

- | | |
|-------------|----------------------------|
| • tools | • interactions |
| • systems | • debugging infrastructure |
| • workflows | • content pipelines |
| • behaviors | • prototyping layers |

Every project becomes a reinvention of the foundation before it can become a realization of the vision.

The industry treats this as normal.

I do not think it is.

04 / The Cost of Complexity

As game worlds grow larger and more interconnected, complexity compounds.

Design intent separates from implementation.

Systems evolve independently.

Content changes faster than tooling can support it.

Simulation and runtime behavior drift apart.

Teams lose the ability to reason about the whole.

The human cost shows up everywhere:

- **designers script instead of design**
- **technical designers glue disconnected systems together**
- **engineers clean up design-driven implementation problems**
- **artists rework assets after systems drift**
- **producers manage uncertainty instead of momentum**

This is not a failure of talent.

It is the cost of forcing modern game development through fragmented pipelines.

05 / The Question

For years, I kept returning to one question:

What if modern game development is not constrained by creativity, but by the architecture beneath the creative process?

If iteration is expensive, then iteration itself needs a better foundation.

If systems drift, then coherence needs to be designed into the process.

If designers depend on engineers for every adjustment, then authored intent needs a safer way to become playable behavior.

If every project starts at zero, then the industry is missing a design-time layer.

That question became Forge.

06 / Why Forge Exists

Forge exists to solve a structural problem in game development.

Not by replacing engines.

Not by replacing designers.

Not by automating creativity.

Forge exists to give teams a design-time engine: a place where world logic, systems, pacing, narrative behavior, simulation, validation, and runtime delivery can stay coherent as complexity grows.

Forge is designed so that:

- designers author intent
- systems remain inspectable
- iteration becomes reversible
- simulation matches runtime
- complexity becomes visible
- engines consume deterministic outputs

The goal is simple:

Teams should spend more time building experiences and less time fighting the pipeline.

Closing

I do not believe modern game development has a talent problem.

I believe it has an infrastructure problem.

Forge is my attempt to solve it.

Forge — a design-time engine for modern game development.

Working repository and full architectural brief are available under NDA.

Elliott@Webb.Systems